HF Model Finder: Constraint-Verified Retrieval-Augmented Generation for Finding Hugging Face Models

Capstone Project Report submitted in partial fulfilment

of

Name of the Programme

in

Name of the Stream

by

Aditya Rane (Roll No., SAP ID)

Under the supervision of

Name of Faculty Mentor

(Designation, Name of the Department, MPSTME)

SVKM's NMIMS University

(Deemed-to-be University)

MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING (MPSTME)

Vile Parle (W), Mumbai-56

(Academic Year)

ABSTRACT

Choosing a pre-trained model on the Hugging Face Hub is a constrained-selection problem. A developer wants "a Marathi named-entity model under 500 MB that I can use commercially", and four of those five requirements are hard constraints (task, language, size and licence rights) that a model either meets or does not. Keyword search cannot read the soft part of such a request. The Hub's own filters match a licence *string* rather than a right. A plain retrieval-augmented generation (RAG) pipeline recommends whatever sounds relevant: in this work it broke at least one stated constraint on half of its recommendations.

This report presents **HF Model Finder**, a constraint-verified RAG system. A language model only parses the request into constraints and later writes a cited explanation; every pass, fail or unknown verdict is decided by deterministic code that reads model metadata, never model output or card text. Missing metadata is shown as an explicit *unknown* instead of being guessed or silently dropped. The system is a LangGraph pipeline with clarifying questions, an editable reading of the request and a prompt-injection defence. It searches 15,991 models across eleven tasks (speech, translation, text, vision, generation, summarization and question answering) with a deliberate emphasis on Indian languages, using hosted multilingual embeddings in Pinecone and a Streamlit interface.

The system was evaluated against naive RAG, BM25 with filters and the Hub's own filters on 100 queries written before any tuning. On 75 normal queries it recommended no model that fails a requested constraint (0.0% against 50.4% for naive RAG and 25.9% to 32.8% for the Hub filters) and 83.3% of its recommendations were fully verified (29.1% for naive RAG). It asked a clarifying question on all 15 vague queries and never claimed verification when no model in the corpus could pass. In a blind relevance study graded by independent language-model judges (not humans), it had the highest mean grade (1.28 of 2). The report states its limits plainly: 27% to 30% of models have unknown size, relevance and faithfulness were judged by language models, and the six added tasks were checked by smoke tests rather than the 100-query study.

Note: the software was developed with the assistance of an AI coding assistant (Claude Code); see Section 3.4. Relevance and faithfulness grades were produced by language-model judges, not humans.

CONTENTS

Chapter 1 Introduction.....1
Chapter 2 Literature Survey.....2
Chapter 3 Methodology, Implementation, Flowchart and Algorithms.....3
Chapter 4 Snapshots, Results and Discussion.....8
Chapter 5 Source Code.....11
Chapter 6 Conclusion and Future Scope.....13
References.....13
Appendix A: Soft Code Flowcharts.....15
Appendix B: Data Sheet of the Snapshot.....15

Chapter 1 Introduction

A model repository is useful only if the right model can be found in it. The Hugging Face Hub hosts more than a million pre-trained models, yet a developer who needs "a small Marathi named-entity model that I can use in a commercial product" must still translate that sentence into task, language, licence and size checks and read model cards one by one. This report describes **HF Model Finder**, which does that translation and checking automatically and is built so that its answers can be trusted on size, licence rights, language and task. Chapter 1 gives the background, motivation, problem and contributions; Chapter 2 surveys the literature and the gap; Chapter 3 describes the design; Chapter 4 shows the application and the results; Chapter 5 lists core code; Chapter 6 concludes. The appendices hold flowcharts and a data sheet.

1.1 Background of the project topic

Retrieval-augmented generation (RAG) fetches documents at question time and gives them to a language model as context: Lewis *et al.* combined a generator with a dense retriever [1], REALM trained retriever and model together [2], Fusion-in-Decoder read many passages at once [3], RETRO retrieved from trillions of tokens [4] and Self-RAG learned when to retrieve [5]; a survey organises the family into naive, advanced and modular RAG [6]. Its building blocks are dense retrieval [7], sentence encoders [8], the lexical BM25 baseline [9], cross-encoder re-ranking [10], multilingual embeddings [11], [12] and approximate nearest-neighbour indexes with filters [13]–[15]. Language models can also act as controllers that call tools or other models [16]–[18]. Model cards and datasheets document models [19], [20], but documentation on public hubs is uneven and licences are often missing or mis-categorised [21], [22]. Indic-language resources are concentrated in a few corpora, benchmarks and models [23]–[27], while most community fine-tunes sit in the low-download long tail.

1.2 Motivation and scope

A request such as "a Marathi NER model under 500 MB, usable commercially, that works well on noisy social-media text" mixes one *soft* preference, which meaning can rank, with several *hard* constraints, which are facts. Keyword search cannot read meaning; the Hub's licence filter matches an identifier, not a right; and plain RAG recommends models whose cards *sound* relevant: in this project's evaluation a naive RAG baseline recommended a model that fails a stated requirement for 50.4% of its recommendations. Metadata is also often missing (about 27% of models in the original corpus have no determinable size). The scope is a *model finder* over a snapshot of 15,991 models in eleven tasks (speech recognition, translation, text and token classification, sentence similarity, image classification, object detection, image segmentation, text generation, summarization, question answering). It does not run models, give legal advice or rank by benchmark scores.

1.3 Problem statement

To design, implement and evaluate a retrieval-augmented system that returns Hugging Face models verified in code against every stated hard constraint (task, language, size, commercial and modification rights); that shows missing metadata as an explicit *unknown*; that asks for clarification when a request is under-specified; and that is measurably better on constraint satisfaction than naive RAG, keyword search with filters and the Hub's own filters, with attention to Indian-language models.

1.4 Salient contributions

1. **"The LLM parses, code verifies":** the language model only extracts constraints and writes explanations; every pass, fail or unknown verdict is computed from metadata.

2. **Unknown as a first-class verdict,** with retrieval filters that keep unknowns and a banner when nothing is fully verified.
3. **Rights, not licence strings** (a 54-licence map) and a reproducible size rule; a **verify-wide, show-narrow** design (100 verified, 5 shown).
4. A **LangGraph** pipeline with clarifying questions, editable reading of the request, a prompt-injection defence, snippets and fine-tuning estimates.
5. A **reproducible evaluation** on 100 queries fixed before tuning, with its limits stated.

1.5 Organization of the report and standards followed

The chapters follow the order in the first paragraph. The project is a student system, so the standards below are used as guides and no formal compliance is claimed: requirements are written as identified, verifiable statements [28]; the design is described in several views [29]; verification activities and test documentation follow the practice of [30] and [31]; and transparency, honesty about unknowns and user autonomy are treated as requirements [32].

Chapter 2 Literature Survey

2.1 Introduction to the overall topic

The topic joins four areas: retrieval-augmented generation [1]–[6], retrieval and ranking [7]–[15], language models that use tools [16]–[18] and the documentation of models and licences [19]–[22], with two cross-cutting concerns: Indic-language coverage [23]–[27] and trustworthy evaluation and security [33]–[40].

2.2 Survey of the literature

Retrieval-augmented generation. Lewis *et al.* [1] coupled a sequence-to-sequence generator with a dense retriever over Wikipedia, fine-tuned them together and set the state of the art on open-domain question answering, with the advantage that the knowledge base can be updated by replacing the index; the retrieved items are free text, so nothing can require them to satisfy a structured condition. REALM [2] put a latent retriever into pre-training and beat earlier knowledge-augmented models by four to sixteen points. Fusion-in-Decoder [3] encodes each passage with the question and lets the decoder attend to all of them, so accuracy keeps improving as more passages are read, which supports retrieving wide. RETRO [4] retrieved from a two-trillion-token database and matched models with 25 times more parameters, showing that a store can replace parameters. Self-RAG [5] learns when to retrieve and critiques its own output, but the critique is a learned judgement and not a guarantee. The survey of Gao *et al.* [6] names naive, advanced and modular RAG; the system here is modular RAG plus a verification stage.

Retrieval, ranking and filtered search. Dense retrieval beat BM25 by nine to nineteen points of top-20 accuracy [7]; Sentence-BERT cut the search for the most similar pair among 10,000 sentences from about 65 hours to five seconds [8]; BM25 remains the standard lexical baseline [9]; and a BERT cross-encoder improved MS MARCO re-ranking by 27% (relative) [10], which is why a re-ranker follows the vector search here. BGE-M3 and multilingual E5 map more than a hundred languages into one space [11], [12]. Filtered-DiskANN [15] builds label filters into the graph index but assumes every item has its labels; production metadata filters simply drop items that lack the field, so missing data must be handled by the application.

Tools and model selection. ReAct interleaves reasoning with actions [16], and PAL has the model write a program that an interpreter runs [17]; the latter is the precedent for letting code compute what a model should only request. HuggingGPT [18] lets a language model choose Hub models by reading their descriptions, with no verification of size, licence or language.

Documentation, licences and Indic resources. Model cards and datasheets [19], [20] are conventions, not schemas. A study of Hugging Face model reuse found missing attributes, claimed-versus-actual discrepancies and model risks [21], and an audit of 1,800+ datasets found licences omitted in over 70% of cases and wrong in over 50% on hosting sites [22]. Indic corpora, benchmarks and models [23], [24] and translation for all 22 scheduled languages [25] exist as research artefacts, and no cited work helps choose among the thousands of community fine-tunes derived from them.

Evaluation and security. RAGAs defines reference-free faithfulness [33]; strong LLM judges agree with humans more than 80% of the time but have biases [34]; BM25 is a robust zero-shot baseline [35]. Retrieved text can carry instructions that hijack an application [38], which is why verdicts here never depend on the language model. Agreement between graders is reported with Cohen's kappa [39] and Landis and Koch's bands [40].

2.3 Comparison of the surveyed work

Table 2.1. Comparison of selected related work with the requirements of constraint-verified model finding.

Work	Provides	Hard constraints guaranteed	Missing metadata	Gap for this problem
RAG, REALM, FiD, RETRO [1]–[4]	Retrieval plus generation over text	no	no	Relevance only
Self-RAG [5]	Adaptive retrieval, self-critique	no	no	Critique is a model judgement
Dense, BM25, re-ranking [7], [9], [10]	Retrieval and ordering	no	n/a	No constraint semantics
Filtered ANN [15]	Search with label filters	for labelled items	assumes present	Unknowns not modelled
ReAct, PAL [16], [17]	Reasoning with tools or code	partly	no	Not a selection method
HuggingGPT [18]	LLM picks Hub models	no	no	Selection by text
Reuse study, licence audit [21], [22]	Evidence of missing, wrong data	n/a	identified	No tool
This work	LLM parses, code verifies	**yes**	**yes**	Evaluated on 100 queries

2.4 Gap analysis and problem definition

Five gaps remain: (1) no selection method guarantees hard constraints; (2) missing metadata is not modelled and needs a verdict of "the data does not say"; (3) licences are displayed, not interpreted as rights; (4) the Indic long tail and untagged multilingual models are invisible to popularity-ranked and tag-filtered search; and (5) RAG metrics measure faithfulness and relevance, not how often a recommendation breaks a stated requirement. They lead to the problem of Section 1.3, with objectives to parse requests into validated constraints, verify each in code with three verdicts, retrieve while keeping unknowns, interpret licences and sizes by fixed rules, serve beginners, and evaluate against three baselines on queries fixed before tuning.

Chapter 3 Methodology, Implementation, Flowchart and Algorithms

3.1 Requirements and design principles

Requirements were written as identified, verifiable statements [28] (Table 3.1). Four principles follow: the language model parses and code verifies; *unknown* is a first-class value; filters are loose and verification is exact; and every claim has a measurement.

Table 3.1. Main requirements and the evidence that verifies them.

ID	The system shall	Verification
FR-1	show how a request was understood, editable by the user	UI test; Fig. 4.1
FR-2	extract constraints and validate every field against a schema	Unit tests; 29-case extraction set
FR-3	ask at most two clarifying questions when the task or language pair is missing	Rate on 15 vague queries
FR-4	decide pass, fail or unknown per requested constraint in code, from metadata only	Unit tests; violation rate
FR-5	never present a model that fails a requested constraint	Violation rate on 75 queries
FR-6	show *unknown* where metadata is missing; never guess or omit	Unknown rate; data-quality report
FR-7	state when no returned model is fully verified	12-query no-verified slice
FR-8	cite only candidates and never state licence, size or language in explanations	Faithfulness; poisoned-card test
NFR	be reproducible (checksummed snapshot, one config, cached calls), treat card text as untrusted, and state its limits	Reproduction run; poison test

3.2 Block diagram

Fig. 3.1 shows the offline pipeline (Hub to index) and the online pipeline (request to answer); they meet only in the index, which holds one vector and the model's metadata per card chunk.

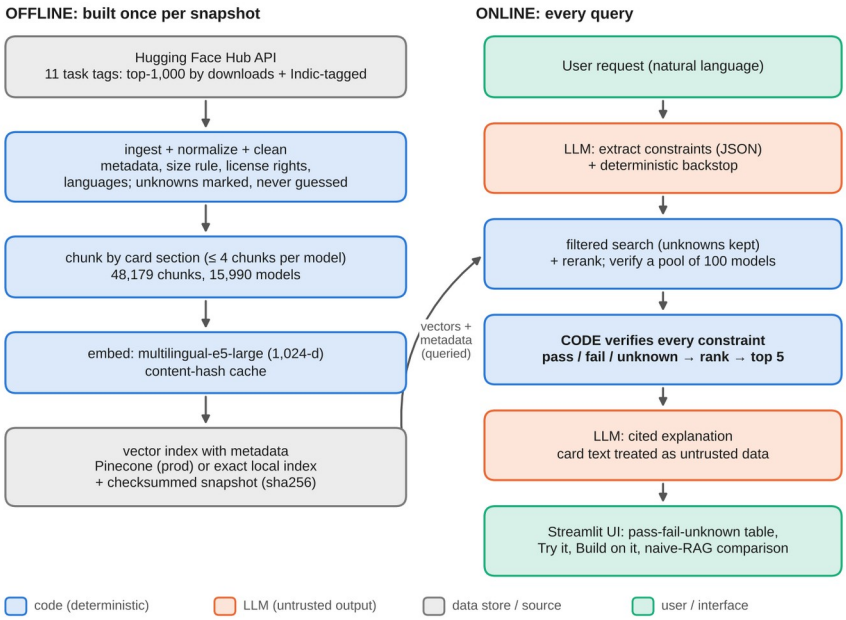


Fig. 3.1. Block diagram: offline data pipeline (left) and online query pipeline (right); orange marks language-model use.

3.3 Hardware description

No special hardware is needed. Development and evaluation ran in a cloud container (4 virtual CPUs, 16 GB RAM, no GPU, Python 3.13); embedding and re-ranking use Pinecone-hosted `multilingual-e5-large` [12] and `bge-reranker-v2-m3` [11] on a serverless index (48,179 vectors, 1,024 dimensions, cosine); the language model is `gpt-oss-20b` [41] on Groq's free tier (8,000 tokens per minute), with `gpt-oss-120b` as fallback and a local Ollama model as an offline option. The hosted re-ranker's 500-requests-a-month quota was reached on 25 October 2026, so uncached searches are not re-ranked and the page says so.

3.4 Software description

The stack is Python, LangGraph [42] for the query graph, Streamlit [43] for the interface, Pinecone for vectors and filters [44], the Hugging Face Hub API [45] for data, and 335 offline pytest tests (about

3,600 lines in `src/` and `app.py`, 900 in `eval/`, 3,300 in `tests/`). One `config.yaml` holds every number that affects results.

Data pipeline. For each of eleven tasks the top 1,000 models by downloads are fetched, plus all models tagged with ten Indic languages for the language tasks; downloads are used only for sampling. Normalisation turns raw records into verifiable facts. *Size* is the exact sum of the top-level `.safetensors` (else `.bin`) files, and unknown if any doubt remains (29.5% of models). *Rights* come from a map of 54 licences to commercial use and modification (*yes*, *no*, *conditional*, *unknown*); non-commercial and no-derivative licences are mapped separately, and copyleft and use-restricted licences are *conditional* and never a clean pass. *Languages* are normalised to ISO codes with a small cited map for multilingual models, and *translation direction* is recorded only for `opus-mt-<src>-<tgt>` names (380 models). Cards are cleaned, split by heading into chunks of at most 450 tokens (at most four per model), embedded, and indexed in Pinecone and in an exact local index with the same filter semantics; a SHA-256-checked snapshot freezes the 15,991 records.

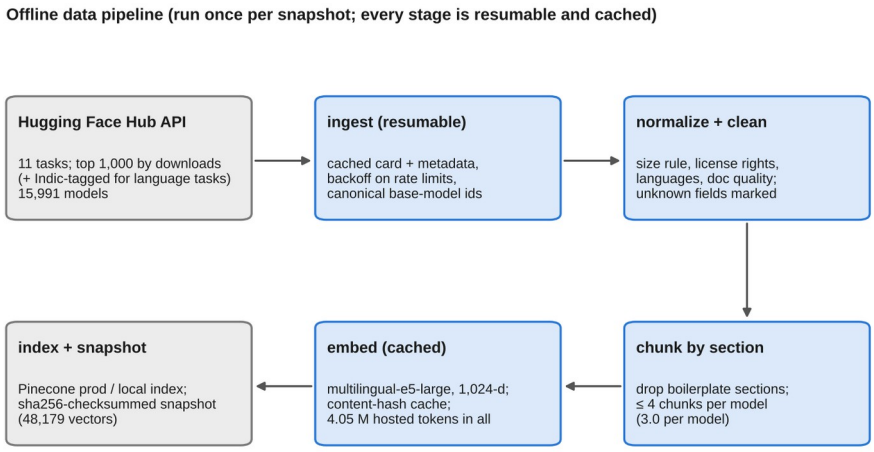


Fig. 3.2. The offline data pipeline: from the Hub API to the index and snapshot (Fig. 3.1 shows it in context).

Query pipeline. A LangGraph graph runs *extract* (deterministic out-of-scope refusal; language-model JSON at temperature 0; validation; a deterministic backstop that fills only what the request plainly states and never runs on instruction-like queries) → *clarify* (at most two questions) → *retrieve* (the user's own words; metadata filter built so unknowns are kept; k doubled from 50 to 400 until 100 distinct models are covered) → *rerank* → *verify* → *relax* or *abstain* → *enrich* (rights notes, lineage, a LoRA/fine-tuning memory estimate of 2P, 2.16P and 16P bytes for P parameters, a template snippet) → *explain*. Verifying a pool of 100 and showing five (Fig. 3.4) raised fully verified answers from 72.3% to 83.3%, because semantic search cannot see size, licence or direction.

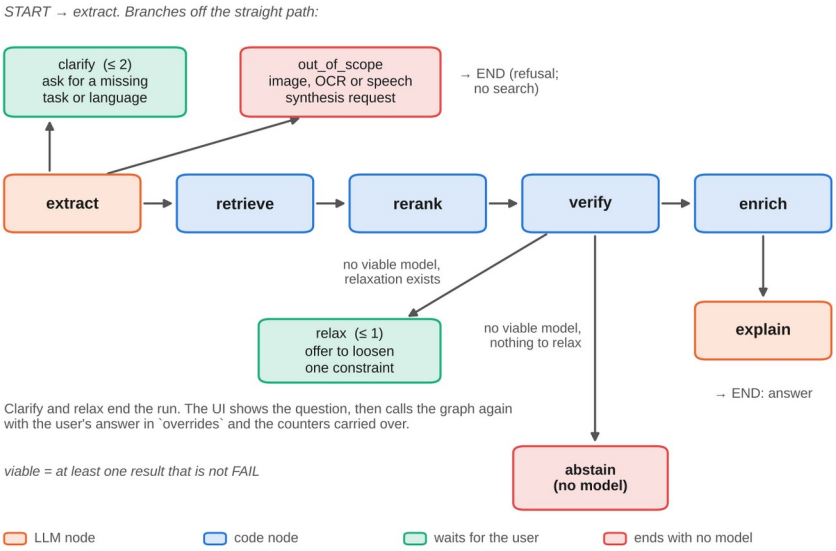


Fig. 3.3. The LangGraph query pipeline; clarify and relax end a run and the interface re-invokes the graph with the user's answer.

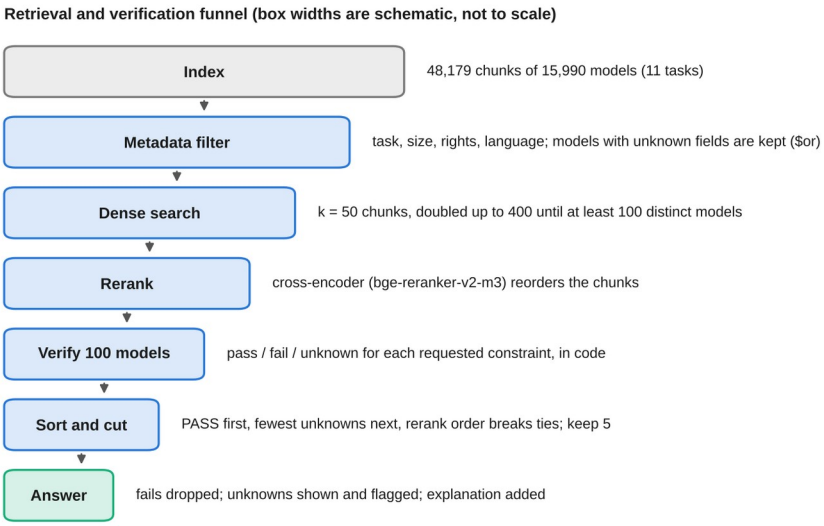


Fig. 3.4. Verify wide, show narrow: from 48,179 chunks to five models.

Algorithm. Algorithm 3.1 gives the verification rule, the only place where a verdict is decided; the flowcharts are in Appendix A.

Algorithm 3.1. Verification of one model and ranking of the pool

```
VERIFY(meta, C):                    # C = constraints the user requested
  task:       UNKNOWN if meta.task empty; else PASS if equal, else FAIL
  languages:  UNKNOWN if list empty/unknown; PASS if all requested present;
              else UNKNOWN if meta.multilingual else FAIL
  direction:  PASS if "src-tgt" in meta.directions else UNKNOWN   # never FAIL
  size:       UNKNOWN if size unknown; PASS if size_mb <= limit else FAIL
  commercial, modifiable:  PASS if yes; FAIL if no; else UNKNOWN
OVERALL = FAIL if any FAIL, else UNKNOWN if any UNKNOWN, else PASS
POOL = 100 best re-ranked models; stable-sort by (OVERALL, #UNKNOWN); drop FAIL; show 5
```

Fig. 3.5 shows how per-constraint verdicts combine. Because a row fails only when the stored field is known and violates the request, a shown model cannot violate a requested constraint, whatever the retriever or the language model does.

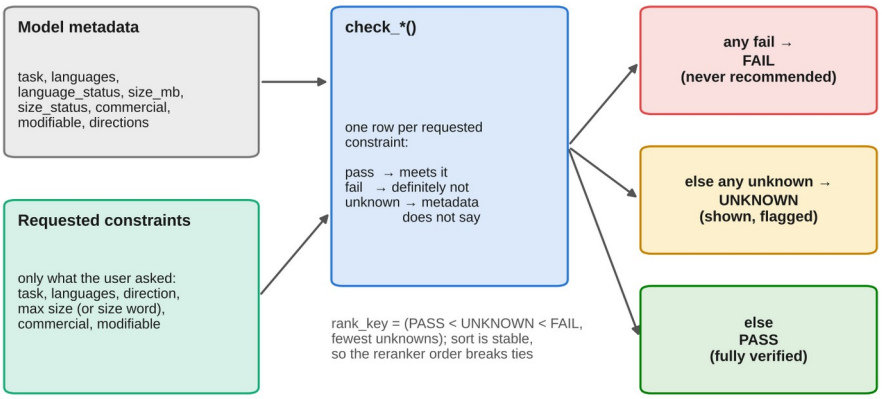


Fig. 3.5. Per-constraint verdicts and their combination into one verdict and a rank.

Security and ethics. Card text is untrusted [38]: verdicts read only metadata, passages are wrapped in `<card_section>` tags with literal tags escaped, the model is told not to follow them or state licence, size or language facts, and citations outside the candidate set are dropped (Fig. 3.6). In line with [32], transparency (every unknown is visible), honesty (rights are *declared*, not legal advice) and user autonomy are requirements.

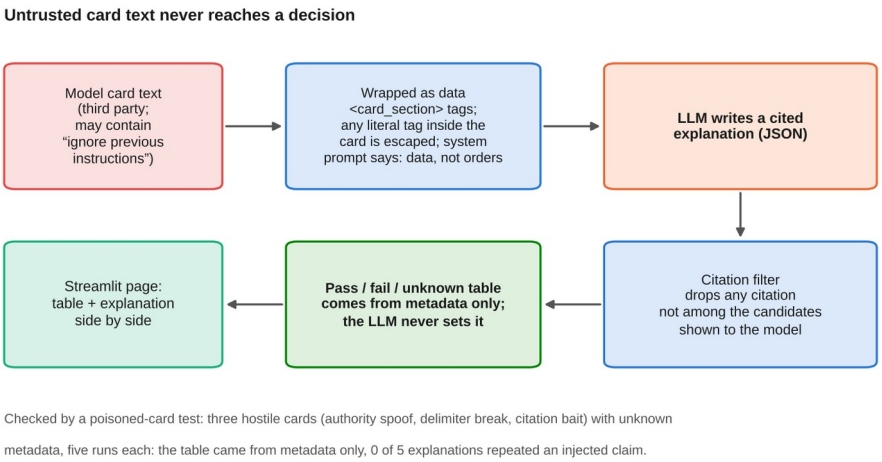


Fig. 3.6. Defence in depth: verdicts never depend on the language model.

Evaluation method. One hundred queries (75 normal, 15 vague, 10 impossible) were written and committed before tuning by one author. Five systems answer them: ours; naive RAG (same index, no filters, no checks); BM25 with the same filters; and the Hub's filters, basic and with `license:apache-2.0`. All are judged by the same verification rules, with outputs frozen before grading. Relevance (0 to 2, 686 blind pairs over 40 queries) and faithfulness (28 explanations) were graded by **language-model judges, not humans**. Testing follows the practice of [30] and [31]: 335 unit tests, a 140-case edge-case catalogue, live-service checks, an extraction test, a poisoned-card test and a Try-it smoke test.

Development process. The project was carried out by one person between 16 and 27 October 2026, with an AI coding assistant (Claude Code) writing and running much of the code and tests under the author's direction; the design decisions, scope and interpretation of results are the author's, and design decisions D1 to D24 were logged with alternatives and rationale. The tuning policy was fixed in

advance: queries committed first, every query used for debugging listed, and no retrieval, prompt or configuration change after outputs were frozen for grading.

Chapter 4 Snapshots, Results and Discussion

4.1 Application snapshots

Fig. 4.1 shows the request "Small sentiment model for Hindi I can use in a commercial app" with the naive-RAG comparison switched on. "We understood" shows that "small" was read as 500 MB. The left column is the full system and every row passes; the right column is naive RAG on the same index with no filters and no checks, and it recommends models whose size row is a red cross. Pass is shown as plain text, fail with a red cross and unknown with a warning sign. For vague requests ("I want a Hindi model") the system asks what the model should do instead of guessing. Fig. 4.2 shows an honest unknown: for "Hindi to Marathi translation under 1 MB" no candidate can be verified (size and direction unknown), so the page shows the banner "No model fully verified" and each row carries a warning sign. Under each model, *Try it* gives a template snippet and *Build on it* gives the modification right, lineage, a GPU-memory estimate for inference, LoRA and full fine-tuning, and a link to the official guide (Fig. 4.2 (b)). Requests the index cannot serve ("text to image diffusion model") are refused with nothing shown.

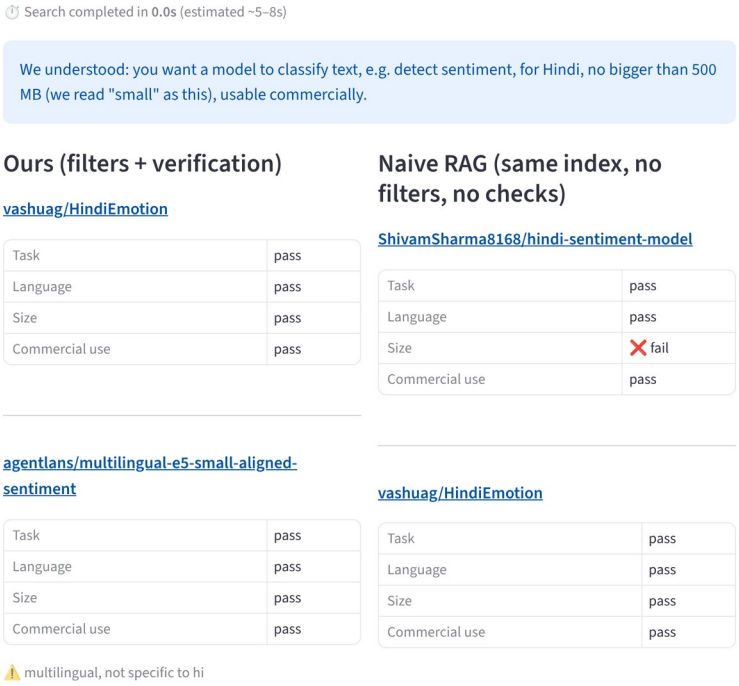
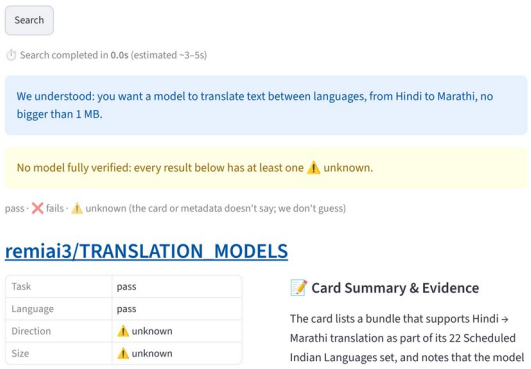
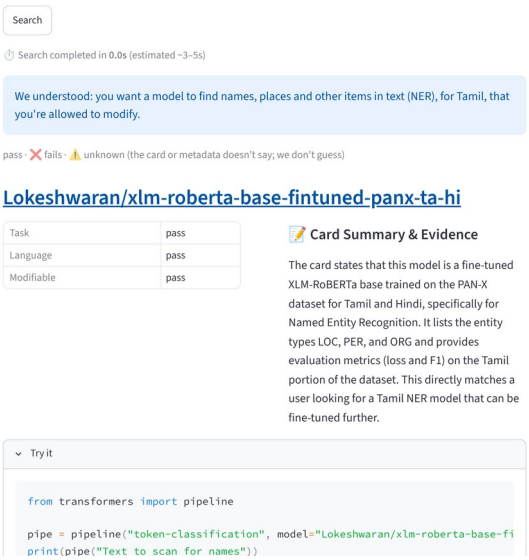


Fig. 4.1. Verified results (left) against naive RAG (right) for a small Hindi sentiment model under a commercial licence.



(a) Banner when nothing is fully verified



(b) Try it and Build on it panels

Fig. 4.2. Unknowns are shown, never guessed; the next step is one click away.

4.2 Corpus and extraction

The snapshot has 15,991 models in eleven tasks. Size is unknown for 29.5% (27.2% in the original five tasks, 6% for image classification and 75% for object detection), commercial use is *yes* for 70.3%, *conditional* for 6.5%, *no* for 3.2% and *unknown* for 20.1%, and 380 translation models state a direction. The language model alone matched the expected constraint record on 90% of 29 test cases with five silent drops; with the deterministic backstop the match was 100% with none (a tuning-set figure, not a generalisation estimate).

4.3 Comparison with the baselines

Table 4.1 gives the main result on the 75 normal queries; every system's top five is judged by the same verification rules.

Table 4.1. Main results on 75 normal queries (top five per query).

System	Answered %	Recs	Violation %	Unknown %	Fully verified %	Median s
Ours (constraint-verified RAG)	97.3	365	**0.0**	16.7	**83.3**	0.22
Naive RAG	100.0	375	50.4	20.5	29.1	0.07
Keyword (BM25) + filters	100.0	368	0.0	62.0	38.0	0.05
HF Hub, basic filters	100.0	375	32.8	13.6	53.6	0.10
HF Hub, licensed filters	100.0	375	25.9	14.7	59.5	0.09

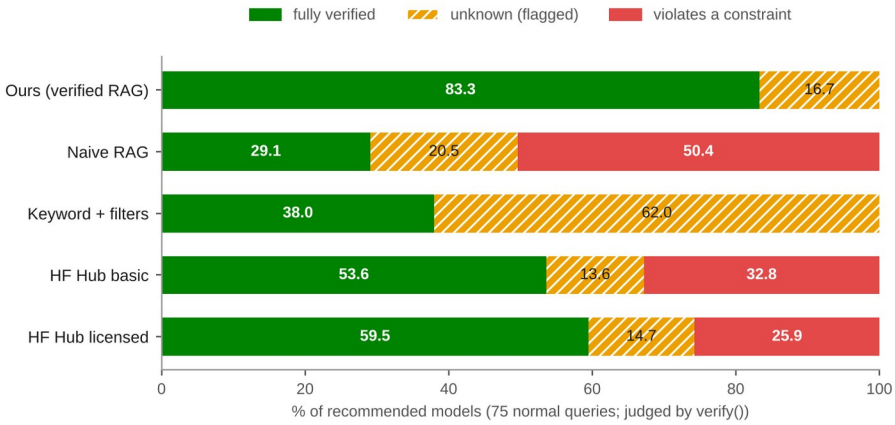


Fig. 4.3. Share of recommended models that are fully verified, unknown or violating, by system.

The full system recommended no failing model and 83.3% of its recommendations were fully verified, against 29.1% for naive RAG and 59.5% for the best Hub filter. Fig. 4.4 shows which constraints the baselines break. It answered 73 of 75 queries; on the other two it asked a clarifying question. Naive RAG failed the size limit on 50.4% of its recommendations, the task on 24.1% and the language on 19.4%. The Hub cannot filter by size and failed it on 80.0% (basic) and 71.9% (licensed) of recommendations; its licensed variant removes commercial-use failures only by accepting a single licence string. Keyword search never violates because it uses the same filters, but 62% of its recommendations are unknown: BM25 matches words, which say nothing about size or licence. Verifying a pool of 100 instead of 25 raised fully verified answers from 72.3% to 83.3% and repaired six translation queries.

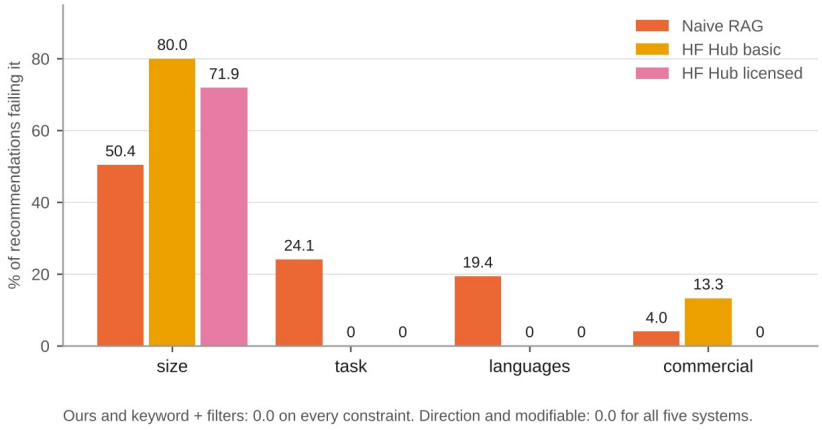


Fig. 4.4. Recommendations that fail each constraint, by baseline (ours and keyword search: 0.0 on every constraint).

On the 15 vague queries the system asked a clarifying question every time; baselines answered all of them. Ten requests written to be impossible could not be proved impossible, because a model with unknown size or language never fails; the abstain and relax paths are therefore tested but unreachable on this corpus. On the 12 queries for which no model in the corpus fully passes, ours and keyword search recommended only unknown models, ours showed the banner on all 12 and never claimed verification, while naive RAG and the Hub filters recommended violating models for 63.3% to 75.0% of their recommendations.

4.4 Relevance, faithfulness, security and speed

Relevance was graded 0 to 2 on a blind sheet of 686 pairs over 40 queries by **language-model judges, not humans** (Table 4.2). The full system has the highest mean grade, so verification did not cost relevance. A repeat pass by different judges over 266 pairs agreed on 92.1% of grades (Cohen's kappa

[39] 0.874 unweighted, 0.945 quadratic-weighted); this shows consistent judging, not agreement with humans.

Table 4.2. Relevance at five (judged by language models, blind).

System	Queries	Models graded	Mean grade	Queries with a grade-2 model	Models graded 1 or 2
Ours	38	190	**1.28**	**81.6%**	**76.8%**
Keyword + filters	40	197	1.06	65.0%	70.1%
Naive RAG	40	200	0.87	67.5%	51.5%
HF Hub, licensed	40	200	0.69	22.5%	61.5%

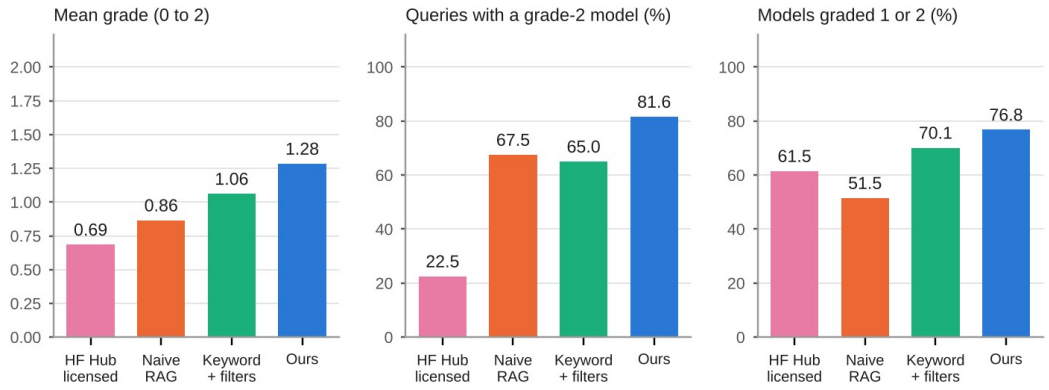


Fig. 4.5. Relevance results from the blind study (language-model judges).

Judges also counted 98 factual claims in 28 explanations: 80 (81.6%) were supported by the shown card text and 14 explanations (50%) were entirely supported, so explanation text is a convenience and not evidence; the verdict tables do not depend on it. With three hostile cards of unknown metadata, tables came from metadata only in 5 of 5 runs and no explanation repeated an injected claim. The Try-it template ran for 20 of 32 small models (MLX, quantised and custom-code models need other tooling). A warm live query took a median of 1.3 s (p95 2.0 s) without the explanation, and 7 to 17 s with it.

4.5 Discussion and limitations

Verification in code removes violations that retrieval alone cannot, without costing relevance, and unknowns are shown instead of guessed. The main limit is the data: where 27% to 30% of sizes, 20% of rights and most translation directions are unknown, no method can verify them, so the 16.7% unknown rate is close to a floor. Other limits: relevance and faithfulness were judged by language models that share a model family with the assistant that helped build the system; one author wrote the queries and built the system; the 100-query evaluation covers the original five tasks and the six added tasks were checked by smoke queries only; the hosted re-ranker's monthly quota is exhausted, so new searches are not re-ranked (verdicts are unaffected); rights are read from *declared* licences and are not legal advice; and the snapshot is a sample of the Hub dated 25 October 2026.

Chapter 5 Source Code

The repository (about 3,600 lines in `src/` and `app.py`, 900 in `eval/`, 3,300 in `tests/`) is at <https://github.com/Adityarane012/RAG-Project>. Everything that decides a verdict is a plain module with no network access; the listings below are the core of `src/verify.py`, copied from the file and line ranges in their captions.

Listing 5.1. src/verify.py (lines 73 to 90): one result per requested constraint.

```
def verify(meta: dict, constraints: dict) -> dict[str, str]:
    """Per-constraint result for one model, for the constraints that were requested."""
    results: dict[str, str] = {}
    if constraints.get("task"):
        results["task"] = check_task(meta, constraints["task"])
    if constraints.get("langs"):
        results["languages"] = check_languages(meta, constraints["langs"])
    if constraints.get("direction"):
        results["direction"] = check_direction(meta, constraints["direction"])
    size_limit = resolve_max_size_mb(constraints.get("max_size_mb"), constraints.get("size_word"))
    if size_limit is not None:
        results["size"] = check_size(meta, size_limit)
    if constraints.get("commercial"):
        results["commercial"] = check_right(meta, "commercial")
    if constraints.get("modifiable"):
        results["modifiable"] = check_right(meta, "modifiable")
    return results
```

Listing 5.2. src/verify.py (lines 106 to 123): combining results and ranking.

```
def overall(results: dict[str, str]) -> str:
    """fail if anything fails, else unknown if anything is unknown, else pass."""
    values = results.values()
    if FAIL in values:
        return FAIL
    if UNKNOWN in values:
        return UNKNOWN
    return PASS


_RANK = {PASS: 0, UNKNOWN: 1, FAIL: 2}


def rank_key(results: dict[str, str]) -> tuple[int, int]:
    """Sort key: full passes first, then fewest unknowns (D12). Python's sort is
    stable, so the reranker's order is kept within each group."""
    n_unknown = sum(1 for v in results.values() if v == UNKNOWN)
    return (_RANK[overall(results)], n_unknown)
```

Listing 5.3. src/retrieve.py (lines 13 to 43): the metadata filter keeps models whose field is unknown.

```
def build_filter(constraints: dict) -> dict:
    """Pinecone metadata filter for the hard constraints in `constraints`."""
    clauses: list[dict] = []

    if constraints.get("task"):
        clauses.append({"task": {"$eq": constraints["task"]}})

    size_limit = resolve_max_size_mb(constraints.get("max_size_mb"), constraints.get("size_word"))
    if size_limit is not None:
        clauses.append({"$or": [
            {"size_mb": {"$lte": size_limit}},
            {"size_status": {"$eq": "unknown"}},
        ]})

    # Exclude only a definite "no"; conditional and unknown go on to verify.py.
    for right in ("commercial", "modifiable"):
        if constraints.get(right):
            clauses.append({right: {"$in": ["yes", "conditional", "unknown"]}})

    # Models from the multilingual map already have their covered languages merged into `languages`.
    for lang in constraints.get("langs") or []:
        clauses.append({"$or": [
            {"languages": {"$in": [lang]}},
            {"language_status": {"$eq": "unknown"}},
            {"multilingual": {"$eq": True}},     # list may be partial; verify decides (pass /
   unknown)
        ]})

    if not clauses:
        return {}
    return clauses[0] if len(clauses) == 1 else {"$and": clauses}
```

Chapter 6 Conclusion and Future Scope

This project built and evaluated HF Model Finder, a constraint-verified RAG system over 15,991 Hugging Face models in eleven tasks. The results support four conclusions. (1) Verification in code removed constraint violations: none on 75 normal queries against 50.4% for naive RAG and 25.9% to 32.8% for the Hub's filters. (2) Honest unknowns beat guesses: 83.3% of recommendations were fully verified, the rest were marked, and the system never claimed verification when no model could pass. (3) Verifying wide and showing narrow matters (72.3% to 83.3% fully verified). (4) Verification did not cost relevance: in a study graded by language-model judges the system had the highest mean grade (1.28 of 2).

Future scope. (1) Validate the judged grades with human annotators on a sample. (2) Extend the evaluation queries to all eleven tasks, with queries from people other than the builder. (3) Reduce unknowns without guessing: parse GGUF and ONNX sizes separately, read custom licence text, extract translation direction from cards. (4) A stronger abstention policy and a re-ranker without a quota. (5) Keep the index fresh with an incremental pipeline and live lookup in check-a-model mode. (6) Add tasks that are currently refused or mapped to the nearest task, and run a user study comparing time to a usable model with the Hub's search.

References

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "Retrieval augmented language model pre-training," in *Proc. 37th Int. Conf. Machine Learning (ICML)*, PMLR vol. 119, 2020, pp. 3929–3938.

[3] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proc. 16th Conf. European Chapter of the Association for Computational Linguistics (EACL)*, 2021, pp. 874–880.

[4] S. Borgeaud *et al.*, "Improving language models by retrieving from trillions of tokens," in *Proc. 39th Int. Conf. Machine Learning (ICML)*, PMLR vol. 162, 2022, pp. 2206–2240.

[5] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2310.11511.

[6] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, Dec. 2023.

[7] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, Dec. 2009.

[10] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, Jan. 2019.

[11] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 2318–2335.

[12] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 text embeddings: A technical report," arXiv:2402.05672, Feb. 2024.

[13] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[14] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.

[15] S. Gollapudi *et al.*, "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in *Proc. ACM Web Conf. 2023 (WWW '23)*, 2023, pp. 3406–3416.

[16] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

[17] L. Gao *et al.*, "PAL: Program-aided language models," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, PMLR vol. 202, 2023, pp. 10764–10799.

[18] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. arXiv:2303.17580.

[19] M. Mitchell *et al.*, "Model cards for model reporting," in *Proc. Conf. Fairness, Accountability, and Transparency*, 2019, pp. 220–229.

[20] T. Gebru *et al.*, "Datasheets for datasets," *Commun. ACM*, vol. 64, no. 12, pp. 86–92, Dec. 2021.

[21] W. Jiang *et al.*, "An empirical study of pre-trained model reuse in the Hugging Face deep learning model registry," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023, pp. 2463–2475.

[22] S. Longpre *et al.*, "A large-scale audit of dataset licensing and attribution in AI," *Nat. Mach. Intell.*, vol. 6, no. 8, pp. 975–987, Aug. 2024.

[23] D. Kakwani *et al.*, "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 4948–4961.

[24] S. Doddapaneni *et al.*, "Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages," in *Proc. 61st Annu. Meeting of the Association for Computational Linguistics (ACL), Vol. 1: Long Papers*, 2023, pp. 12402–12426.

[25] J. Gala *et al.*, "IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages," arXiv:2305.16307, May 2023.

[26] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, PMLR vol. 202, 2023, pp. 28492–28518.

[27] A. Conneau *et al.*, "Unsupervised cross-lingual representation learning at scale," in *Proc. 58th Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 8440–8451.

[28] *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE Standard 29148:2018, 2018.

[29] *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*, IEEE Standard 1016-2009, 2009.

[30] *IEEE Standard for System, Software, and Hardware Verification and Validation*, IEEE Standard 1012-2016, 2016.

[31] *Software and Systems Engineering — Software Testing — Part 3: Test Documentation*, ISO/IEC/IEEE Standard 29119-3:2021, 2021.

[32] *IEEE Standard Model Process for Addressing Ethical Concerns During System Design*, IEEE Standard 7000-2021, 2021.

[33] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, pp. 150–158.

[34] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, vol. 36, Datasets and Benchmarks Track, 2023. arXiv:2306.05685.

[35] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogenous benchmark for zero-shot evaluation of information retrieval models," in *Proc. 35th Conf. Neural Information Processing Systems, Datasets and Benchmarks Track*, 2021. arXiv:2104.08663.

[36] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, 2024.

[37] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, Mar. 2023.

[38] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, 2023, pp. 79–90.

[39] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, Apr. 1960.

[40] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, Mar. 1977.

[41] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," arXiv:2508.10925, Aug. 2025.

[42] LangChain, Inc., "LangGraph documentation." [Online]. Available: https://langchain-ai.github.io/langgraph/ (accessed Oct. 27, 2026).

[43] Snowflake Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io/ (accessed Oct. 27, 2026).

[44] Pinecone Systems, Inc., "Filter by metadata," Pinecone Documentation. [Online]. Available: https://docs.pinecone.io/guides/search/filter-by-metadata (accessed Oct. 27, 2026).

[45] Hugging Face, Inc., "Hub API endpoints," Hugging Face Hub Documentation. [Online]. Available: https://huggingface.co/docs/hub/api (accessed Oct. 27, 2026).

Appendix A: Soft Code Flowcharts

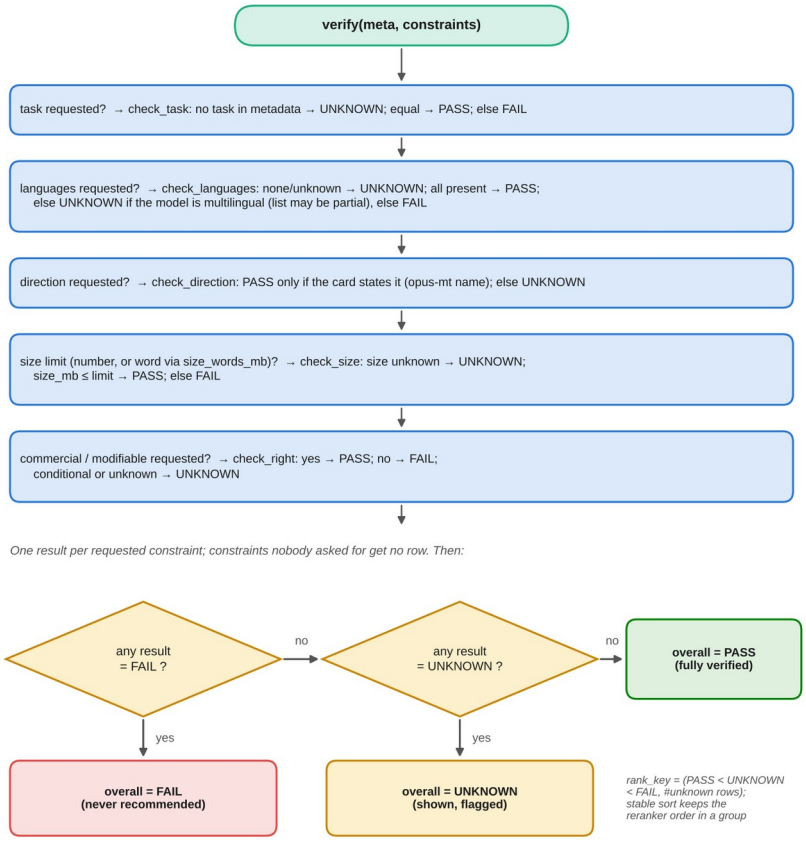


Fig. A.1. Flowchart of `verify`, `overall` and `rank_key`: one result per requested constraint, then the combination.

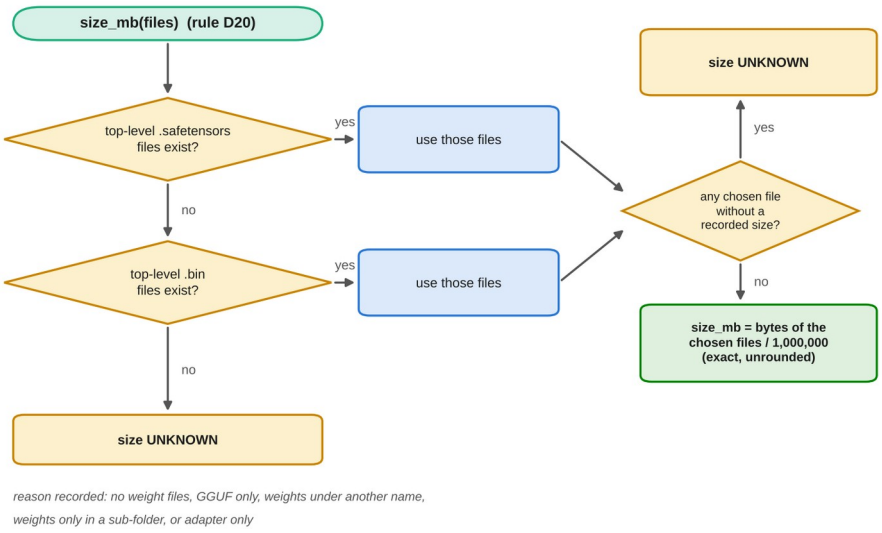


Fig. A.2. Flowchart of the size rule: the exact sum of the main weight files, unknown at any doubt.

Appendix B: Data Sheet of the Snapshot

The snapshot (SHA-256 `45814b8c`…, 25 October 2026) holds one record per Hub model: identifier, task, downloads, languages and status, size and status, parameters, commercial and modification rights, base model, directions and documentation quality. For each task the top 1,000 models by downloads

were fetched, plus all models tagged with ten Indic languages for the language tasks (two transient fetch failures). Size, rights, languages and direction follow the rules of Section 3.4; unknown is preserved. It is intended for verifying constraints, check-a-model and evaluation, and not for ranking quality or legal advice.

Table B.1. Composition of the snapshot by task.

Task	Models	Size unknown	Commercial *yes*	Commercial *unknown*
automatic-speech-recognition	3,469	993	2,989	372
translation	1,886	812	1,241	290
text-classification	1,617	355	1,096	388
token-classification	1,409	284	1,200	156
sentence-similarity	1,383	208	1,006	284
text-generation	1,135	396	668	295
question-answering	1,077	263	655	355
summarization	1,015	130	571	407
image-classification	1,000	62	793	153
object-detection	1,000	746	479	195
image-segmentation	1,000	463	544	314
All tasks	**15,991**	**4,712**	**11,242**	**3,209**